

RELATÓRIO TÉCNICO DE ARQUITETURA DE SOFTWARE

Experiência Prática I — Mapeando os Bastidores da Web: Anatomia de uma API

Disciplina: Desenvolvimento Back-end

Função: Analista Júnior de Arquitetura

Aplicação investigada: Plataforma Maeve

URL: <https://app.ask-maeve.com/br>

Ano: 2026

Objetivo do relatório

Consolidar, em um único documento de referência, os fundamentos teóricos, a preparação, a investigação de requisições HTTP, a análise de contratos JSON, o fluxo cliente-servidor, o resumo executivo e a autoavaliação desenvolvidos durante a primeira experiência prática.

SUMÁRIO

1. Visão geral e objetivos da experiência
2. Fundamentos teóricos essenciais
3. Definição do objeto de estudo: Maeve
4. Mapeamento da arquitetura cliente-servidor
5. Investigação de rede e requisições HTTP
6. Estruturação de dados e contratos de API
7. Fluxograma do ciclo de comunicação
8. Resumo executivo da arquitetura
9. Checklist e autoavaliação
10. Reflexão final e próximos passos

1. VISÃO GERAL E OBJETIVOS DA EXPERIÊNCIA

A experiência prática foi concebida para conectar os fundamentos de Desenvolvimento Back-end à observação de uma aplicação web real. O foco está na engenharia reversa controlada da comunicação entre navegador e servidor, com atenção a arquitetura cliente-servidor, HTTP, APIs REST, endpoints, códigos de status, cabeçalhos e JSON.

Objetivos da experiência

- Compreender, analisar e representar o funcionamento de aplicações back-end e APIs REST.
- Aplicar os conceitos fundamentais de arquitetura cliente-servidor e comunicação HTTP.
- Interpretar a estrutura de requisições e respostas.
- Identificar endpoints e compreender a organização inicial de sistemas server-side.

Rota de aprendizagem

Etapas	Foco	Entregável
1	Preparação e análise inicial	Contextualização e arquitetura base
2	Investigação de requisições e integrações	Métodos, endpoints, headers, status e JSON
3	Documentação e fluxograma	Tabela técnica e fluxo cliente-servidor
4	Síntese e reflexão	Relatório, checklist e autoavaliação

2. FUNDAMENTOS TEÓRICOS ESSENCIAIS

Back-end

É a camada server-side responsável pelo processamento das regras de negócio, manipulação e persistência de dados, autenticação, integrações externas e geração de respostas para o cliente.

Front-end

É a camada visível e interativa executada no cliente. Na experiência analisada, inclui a renderização da interface, captura de eventos, validações preliminares e apresentação dos dados recebidos da API.

Arquitetura cliente-servidor

O cliente envia uma requisição; o servidor interpreta, valida, processa e retorna uma resposta. A separação favorece manutenção, centralização da lógica e escalabilidade.

HTTP e HTTPS

HTTP define regras para comunicação web. HTTPS acrescenta uma camada de proteção baseada em TLS, permitindo que o tráfego seja criptografado.

Principais métodos HTTP

Método	Finalidade	Observação
GET	Recuperação/leitura de recursos	Idempotente
POST	Criação ou submissão de dados	Não idempotente em geral
PUT	Atualização integral de recurso	Idempotente
PATCH	Atualização parcial	Não necessariamente idempotente
DELETE	Remoção de recurso	Idempotente

Códigos de status HTTP

Classe	Categoria	Exemplos / sentido
1xx	Informativos	Processamento/continuidade
2xx	Sucesso	Ex.: 200 OK, 201 Created, 202 Accepted, 204 No Content
3xx	Redirecionamento	Ex.: 301, 302, 304
4xx	Erro do cliente	Ex.: 400, 401, 403, 404
5xx	Erro do servidor	Ex.: 500, 502, 503

3. DEFINIÇÃO DO OBJETO DE ESTUDO: PLATAFORMA MAEVE

A aplicação selecionada para a investigação foi a Plataforma Maeve, apresentada no material como uma aplicação de tecnologia educacional (EdTech) orientada à aprendizagem acelerada e retenção de conteúdo com apoio de Inteligência Artificial Generativa.

Proposta de valor

Converter materiais brutos de estudo — como PDFs, apresentações, documentos e gravações — em artefatos estruturados de aprendizagem, incluindo resumos, simulados e flashcards organizados por repetição espaçada.

Principais ações do usuário

- Ingestão de material didático por upload ou links.
- Estudo ativo com flashcards e repetição espaçada.
- Consulta a resumos automatizados e tutoria com IA (MaeveGPT).
- Gerenciamento de calendário de provas e acompanhamento de métricas.

Arquitetura de processamento apresentada no material

[Upload do Usuário] → [API Gateway] → [Fila de Mensagens / Celery]

↙ Serviço de Extração (OCR/Texto) ↘ Serviço de IA / Prompt Engineering

→ Banco de Dados Relacional / Vetorial ← Geração de Resumos & Flashcards

O material descreve processamento assíncrono para tarefas pesadas, integração com modelos de linguagem, persistência de dados e entrega dos artefatos ao front-end.

4. MAPEAMENTO DA ARQUITETURA CLIENTE-SERVIDOR

A análise separa responsabilidades entre a camada do cliente e a camada do servidor. Essa divisão permite que a interface se concentre na experiência do usuário enquanto o back-end centraliza processamento, regras de negócio, segurança e persistência.

Front-end

- Renderização dinâmica da interface e interatividade.
- Animação de inversão dos flashcards e atualização visual do progresso.
- Validação preliminar de extensões e tamanho de arquivos.
- Serialização dos dados e montagem de requisições HTTP.
- Envio de cabeçalhos de autorização quando necessários.

Back-end

- Autenticação e autorização.
- Recebimento e roteamento das requisições.
- Processamento assíncrono de uploads, OCR e extração de texto.
- Integração com APIs de modelos de linguagem.
- Execução de regras de negócio e algoritmo de repetição espaçada.
- Persistência em bancos de dados relacionais e vetoriais.

Camadas lógicas identificadas

Camada	Responsabilidade
Apresentação / Rotas	Recebe requisições e direciona para o tratamento adequado.
Serviço / Negócio	Aplica regras, cálculos e comportamentos do domínio.
Persistência / Dados	Consulta e grava informações em bancos de dados.
Infraestrutura	Segurança, logging, configuração e integrações externas.

5. INVESTIGAÇÃO DE REDE E REQUISIÇÕES HTTP

A investigação foi orientada pelo uso das DevTools do navegador, especialmente a aba Network/Rede com filtro Fetch/XHR. O roteiro proposto consiste em executar ações na plataforma, selecionar requisições relevantes e registrar método, endpoint, status, headers e payloads.

Procedimento de inspeção

- Abrir a plataforma Maeve.
- Abrir DevTools com F12 ou Ctrl + Shift + I.
- Selecionar Network/Rede e filtrar Fetch/XHR.
- Executar uma ação relevante, como abrir um documento ou acessar Flashcards.
- Examinar Headers, Payload e Response/Preview.
- Registrar evidências e organizar as requisições em tabela.

Requisições centrais documentadas

#	Método	Endpoint	Status	Finalidade
1	GET	/api/documents/{id}/summary	200	Recuperação do resumo e metadados do documento.
2	GET	/storage/v1/object/sign/songs/...	200	Recuperação de áudio por URL assinada.
3	GET	/api/documents/{id}/flashcards	200	Carga do deck de flashcards e dados de repetição.
4	POST	/api/flashcards/review	200 / 201	Registro do feedback da revisão e atualização do progresso.
5	POST	/api/v1/materials/upload	202	Recebimento do arquivo e enfileiramento do processamento assíncrono.

Observação metodológica

O próprio material diferencia hipóteses de arquitetura de evidências de tráfego. Os endpoints acima foram consolidados na atividade como requisições centrais; quando a documentação usa termos como “hipotético” em etapas preparatórias, eles devem ser entendidos como hipóteses até confirmação por captura real.

6. ESTRUTURAÇÃO DE DADOS E CONTRATOS DE API

Na arquitetura analisada, a API funciona como contrato de comunicação entre o front-end e os serviços do back-end. Esse contrato padroniza o formato dos dados e permite que a interface permaneça desacoplada da implementação interna.

Exemplo de payload JSON do flashcard

```
{
  "cardId": "fc-9812-4a",
  "documentId": "5415f02f-51c1-4a1e-ba29-0c866a309b3a",
  "front": "Qual é a composição da Seguridade Social prevista no art. 194 da CF/88?",
  "back": "A Seguridade Social é composta pelo tripé: Saúde (universal), Previdência Social (contributiva) e Assistência Social (não contributiva)",
  "intervalDays": 4,
  "easeFactor": 2.5,
  "reviewed": false
}
```

Campo	Tipo	Função
front	String	Texto da pergunta/estímulo exibida no cartão.
intervalDays	Number	Número de dias calculado para a próxima revisão.
reviewed	Boolean	Indica se o cartão foi respondido no ciclo atual.
cardId	String	Identifica o cartão.
documentId	String	Vincula o cartão ao documento de origem.
easeFactor	Number	Fator de facilidade usado no algoritmo de repetição.

Princípios REST destacados

- Recursos representados por substantivos nas URLs.
- Uso dos métodos HTTP para indicar a operação.
- Stateless: cada requisição deve conter as informações necessárias ao seu processamento.
- Interface uniforme e previsível.
- Possibilidade de cache e existência de camadas intermediárias.
- REST é tratado como estilo arquitetural, não como protocolo rígido.

7. FLUXOGRAMA DO CICLO DE COMUNICAÇÃO

O fluxo documentado utiliza como interação-chave a abertura da aba Flashcards para recuperar o deck de um documento.

ETAPA	FLUXO
1. Evento	Usuário → Front-end: clica na aba "Flashcards".
2. Requisição	Front-end → API: GET /api/documents/{id}/flashcards; Authorization: Bearer <token>; Accept: application/json
3. Segurança	Back-end: valida autenticação e autorização do token.
4. Persistência	Back-end → Banco: consulta cards associados ao Document ID e User ID.
5. Regras	Back-end: aplica a lógica de repetição espaçada e identifica dados relevantes para revisão.
6. Resposta	API → Front-end: HTTP 200 OK; Content-Type: application/json; payload com objetos de flashcards.
7. Desserialização	Front-end: interpreta o JSON recebido.
8. Renderização	Front-end → Usuário: exibe os flashcards interativos no DOM.

Representação resumida

USUÁRIO → FRONT-END → HTTPS/GET → API MAEVE → AUTENTICAÇÃO → BANCO DE DADOS → REGRAS/SM-2 → JSON/200 OK
→ FRONT-END → DOM/FLASHCARDS

8. RESUMO EXECUTIVO DA ARQUITETURA

A investigação técnica na plataforma Maeve documentou o ciclo de vida de uma interação web moderna: a recuperação e renderização do deck de flashcards de um documento previamente processado.

1. Disparo no cliente

O ciclo começa quando o estudante acessa um documento e seleciona a aba "Flashcards". O front-end captura o evento e prepara uma chamada assíncrona.

2. Tráfego HTTP/HTTPS

O cliente envia uma requisição GET ao endpoint `/api/documents/{id}/flashcards`, utilizando um identificador de documento no caminho e cabeçalhos como `Authorization` e `Accept`.

3. Processamento server-side

O back-end valida a autenticação, consulta os registros relacionados ao usuário e ao documento e aplica regras de negócio associadas à repetição espaçada.

4. Resposta padronizada

O servidor retorna 200 OK e um payload JSON contendo objetos estruturados do deck de flashcards.

5. Desserialização e DOM

O navegador interpreta o JSON, converte os dados em estruturas manipuláveis em memória e atualiza a interface para exibir os cartões.

Síntese arquitetural

A principal conclusão da atividade é que a experiência visual do usuário depende de uma cadeia desacoplada de serviços: interface, comunicação segura, API, autenticação, persistência, regras de negócio e serialização de dados. Essa cadeia permite compreender o back-end como a camada que coordena a lógica invisível que sustenta a aplicação.

9. CHECKLIST E AUTOAVALIAÇÃO

Checklist de conformidade técnica

- ☒ Aplicação web real selecionada e contexto de negócio delimitado.
- ☒ Responsabilidades de front-end e back-end segregadas.
- ☒ DevTools / Network utilizados para investigação.
- ☒ Endpoints, métodos HTTP e códigos de status mapeados.
- ☒ Papel das APIs demonstrado e payload JSON analisado.
- ☒ Fluxograma do ciclo cliente-servidor estruturado.
- ☒ Resumo executivo redigido com vocabulário técnico.

Nível de competências registrado na atividade

Competência	Nível
Compreensão da arquitetura cliente-servidor	Excelente
Interpretação de requisições e respostas HTTP	Excelente
Identificação de métodos HTTP e códigos de status	Excelente
Entendimento prático de APIs	Excelente
Capacidade de análise técnica de aplicações web	Excelente
Comunicação técnica e documentação	Excelente

10. REFLEXÃO FINAL E PRÓXIMOS PASSOS

A realização desta atividade representou um salto qualitativo na formação em TI, marcando a passagem de uma visão predominantemente voltada ao uso das aplicações para uma perspectiva analítica e estrutural de desenvolvimento back-end. A análise da plataforma Maeve funcionou como um laboratório prático para relacionar arquitetura distribuída, semântica HTTP e contratos de dados.

Pontos fortes

Destaca-se a compreensão da separação de responsabilidades entre cliente e servidor, especialmente a necessidade de desacoplar processos intensivos, como OCR e consumo de modelos de IA. Também foi consolidada a interpretação de payloads JSON e a correlação entre métodos HTTP e códigos de status, além da capacidade de representar o fluxo em documentação técnica.

Oportunidades de melhoria

Como próximos pontos de aprofundamento, o material registra a necessidade de ganhar maior agilidade na inspeção avançada com DevTools, incluindo análise de CORS, cache e ciclos de tokens JWT, além de avançar para a implementação prática de servidores, persistência e mensageria assíncrona.

Contribuição profissional

A experiência consolida fundamentos importantes de engenharia de software no back-end e amplia o vocabulário técnico para discutir arquitetura, comunicação HTTP, APIs RESTful, segurança, desempenho e decisões de design de sistemas.

Conclusão

A primeira experiência prática foi organizada como um ciclo completo de investigação: selecionar o sistema, compreender sua arquitetura, observar o tráfego, interpretar os contratos de dados, representar o fluxo e refletir sobre as competências desenvolvidas. O documento passa a funcionar como base de consulta para as próximas atividades da disciplina.

Fonte-base: material fornecido pelo estudante no arquivo de apoio desta atividade. O relatório preserva a terminologia e as conclusões apresentadas no material, distinguindo hipóteses arquiteturais de requisições documentadas.